

Practico 01 – Maquinas de Turing

Computabilidad y Complejidad (Teoría de la Computación II)

FCEFQyN – Universidad Nacional de Río Cuarto

Primer Cuatrimestre de 2026

Resolucion completa del Practico 01. Apoyada en el Capítulo 3 del libro de M. Sipser, *Introduction to the Theory of Computation*, y en el material teórico de la cátedra ([Diag.pdf](#), [MaqTuring.pdf](#)).

Índice

Introducción	1
1. Ejercicio 1: lectura del Capítulo 3	2
2. Ejercicio 2: configuraciones de M_2	2
3. Ejercicio 3: deciders por marcado y emparejamiento	5
4. Ejercicio 4: cinta doblemente infinita	9
5. Ejercicio 5: subconjunto infinito decidable	10
6. Ejercicio 6: MT con input read-only	11
7. Parte de programación	12
Conclusiones	14

Introducción

El presente trabajo resuelve la totalidad de los ejercicios del Práctico 01 de la asignatura, dedicado al modelo de Máquinas de Turing (MT), sus variantes y los conceptos básicos de *decidibilidad* y *reconocibilidad*. La guía teórica utilizada es:

- el Capítulo 3 del libro de Sipser ([chapter-03.pdf](#)), que introduce la definición formal de MT (Definición 3.3), sus ejemplos clásicos (M_1, M_2, M_3, M_4), las variantes (multicinta, no determinista, doblemente infinita) y el resultado de robustez (Teoremas 3.13, 3.16);
- las láminas de la cátedra ([Diag.pdf](#)), que tratan cardinalidad, conjuntos contables, diagonalización de Cantor, existencia de lenguajes no computables y el problema de la terminación.

Convenciones de notación.

- Σ es el alfabeto de entrada y $\Gamma \supseteq \Sigma \cup \{\sqcup\}$ el alfabeto de cinta. El símbolo $\sqcup$ denota el blanco.
 - Una *configuración* se escribe uqv , con $u, v \in \Gamma^*$ y $q \in Q$, e indica que el contenido de la cinta es uv (seguido de blancos), el estado actual es q y el cabezal está sobre el primer símbolo de v .
 - $L(M)$ es el lenguaje reconocido por M .
 - “Decidible” = Turing-decidible (alguna MT lo decide); “reconocible” = Turing-reconocible (alguna MT lo reconoce).
-

1. Ejercicio 1: lectura del Capítulo 3

El primer ítem del práctico consiste en *leer el Capítulo 3 del libro de Sipser*. Se asume realizado como prerrequisito teórico para el resto de los ejercicios. Las nociones que se utilizan a partir del Ejercicio 2 incluyen:

- **Definición formal (Sipser 3.3):** una MT es una 7-tupla

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

con Q conjunto finito de estados, Σ alfabeto de entrada (sin el blanco), $\Gamma \supseteq \Sigma \cup \{\sqcup\}$ alfabeto de cinta y función de transición $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$.

- **Reconocibilidad y decidibilidad** (Definiciones 3.5–3.6).
- **Variantes (Sección 3.2):** máquinas multicinta, no deterministas, doblemente infinitas, todas equivalentes a la MT estándar (Teoremas 3.13, 3.16, Problema 3.11).
- **Enumeradores (Sección 3.3, Teorema 3.21):** L es reconocible si y sólo si existe un enumerador que enumera L .
- **Tesis de Church-Turing:** la igualdad informal entre “algoritmo” y “MT” justifica describir máquinas a nivel implementación o de alto nivel.

Cada ejercicio que sigue cita explícitamente las definiciones y resultados del capítulo que invoca.

2. Ejercicio 2: configuraciones de M_2

Enunciado

Para la máquina de Turing M_2 dada en clases (Ejemplo 3.7 de Sipser), dar la secuencia de configuraciones que recorre con las entradas: (a) 0, (b) 00, (c) 000, (d) 000000.

La máquina M_2

M_2 es la MT que decide

$$A = \{0^{2^n} \mid n \geq 0\},$$

es decir, las cadenas de ceros cuya longitud es una potencia de 2. Su descripción de alto nivel es:

$M_2 =$ “Con entrada w :

1. Recorrer la cinta de izquierda a derecha tachando cada segundo 0.
2. Si en la etapa 1 quedó un solo 0, aceptar.
3. Si en la etapa 1 quedaron más de un 0 y la cantidad fue impar, rechazar.
4. Volver al inicio de la cinta.
5. Volver a la etapa 1.”

Cada iteración divide a la mitad la cantidad de 0s; si la longitud no es potencia de 2, en alguna iteración la cantidad será impar y M_2 rechazará; si es potencia de 2, terminará con un único cero y aceptará.

Definición formal

$$M_2 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}}),$$

con $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$, $\Sigma = \{0\}$, $\Gamma = \{0, x, \sqcup\}$, y δ dada por (Figura 3.8 de Sipser):

Estado	0	x	$\sqcup$
q_1	$(q_2, \sqcup, R)$	$(q_{\text{reject}}, x, R)$	$(q_{\text{reject}}, \sqcup, R)$
q_2	(q_3, x, R)	(q_2, x, R)	$(q_{\text{accept}}, \sqcup, R)$
q_3	$(q_4, 0, R)$	(q_3, x, R)	$(q_5, \sqcup, L)$
q_4	(q_3, x, R)	(q_4, x, R)	$(q_{\text{reject}}, \sqcup, R)$
q_5	$(q_5, 0, L)$	(q_5, x, L)	$(q_2, \sqcup, R)$

Intuitivamente, q_1 borra el primer 0 (lo reemplaza por blanco, que funciona como sentinela del extremo izquierdo); q_2 , q_3 y q_4 se alternan para tachar uno de cada dos ceros; q_5 vuelve al extremo izquierdo para reiniciar el barrido.

Convención para escribir configuraciones

Usamos la convención uqv de Sipser: el cabezal está sobre el primer símbolo de v y el contenido completo de la cinta es uv seguido de blancos.

(a) Entrada 0

$$\begin{array}{ll}
C_0 = q_1 0 & \\
C_1 = \sqcup q_2 \sqcup & \delta(q_1, 0) = (q_2, \sqcup, R) \\
C_2 = \sqcup \sqcup q_{\text{accept}} \sqcup & \delta(q_2, \sqcup) = (q_{\text{accept}}, \sqcup, R)
\end{array}$$

Resultado: acepta. Como $|0| = 1 = 2^0$, la entrada pertenece a A .

(b) Entrada 00

$C_0 = q_1 00$	
$C_1 = \sqcup q_2 0$	$\delta(q_1, 0) = (q_2, \sqcup, R)$
$C_2 = \sqcup x q_3 \sqcup$	$\delta(q_2, 0) = (q_3, x, R)$
$C_3 = \sqcup q_5 x$	$\delta(q_3, \sqcup) = (q_5, \sqcup, L)$
$C_4 = q_5 \sqcup x$	$\delta(q_5, x) = (q_5, x, L)$
$C_5 = \sqcup q_2 x$	$\delta(q_5, \sqcup) = (q_2, \sqcup, R)$
$C_6 = \sqcup x q_2 \sqcup$	$\delta(q_2, x) = (q_2, x, R)$
$C_7 = \sqcup x \sqcup q_{\text{accept}} \sqcup$	$\delta(q_2, \sqcup) = (q_{\text{accept}}, \sqcup, R)$

Resultado: acepta. Como $|00| = 2 = 2^1$, pertenece a A .

(c) Entrada 000

$C_0 = q_1 000$	
$C_1 = \sqcup q_2 00$	$\delta(q_1, 0) = (q_2, \sqcup, R)$
$C_2 = \sqcup x q_3 0$	$\delta(q_2, 0) = (q_3, x, R)$
$C_3 = \sqcup x 0 q_4 \sqcup$	$\delta(q_3, 0) = (q_4, 0, R)$
$C_4 = \sqcup x 0 \sqcup q_{\text{reject}} \sqcup$	$\delta(q_4, \sqcup) = (q_{\text{reject}}, \sqcup, R)$

Resultado: rechaza. La cantidad de 0s es 3, impar y mayor que 1; M_2 detecta esto al estar en q_4 y leer un blanco (“fin de pasada con cantidad impar”).

(d) Entrada 000000

Esta entrada genera una traza más larga porque M_2 realiza dos pasadas completas antes de rechazar. La primera pasada deja tres ceros a procesar; la segunda detecta que 3 es impar.

$C_0 = q_1 000000$	
$C_1 = \sqcup q_2 00000$	$\delta(q_1, 0) = (q_2, \sqcup, R)$
$C_2 = \sqcup x q_3 0000$	$\delta(q_2, 0) = (q_3, x, R)$
$C_3 = \sqcup x 0 q_4 000$	$\delta(q_3, 0) = (q_4, 0, R)$
$C_4 = \sqcup x 0 x q_3 00$	$\delta(q_4, 0) = (q_3, x, R)$
$C_5 = \sqcup x 0 x 0 q_4 0$	$\delta(q_3, 0) = (q_4, 0, R)$
$C_6 = \sqcup x 0 x 0 x q_3 \sqcup$	$\delta(q_4, 0) = (q_3, x, R)$
$C_7 = \sqcup x 0 x 0 q_5 x$	$\delta(q_3, \sqcup) = (q_5, \sqcup, L)$
$C_8 = \sqcup x 0 x q_5 0 x$	$\delta(q_5, x) = (q_5, x, L)$
$C_9 = \sqcup x 0 q_5 x 0 x$	$\delta(q_5, 0) = (q_5, 0, L)$
$C_{10} = \sqcup x q_5 0 x 0 x$	$\delta(q_5, x) = (q_5, x, L)$
$C_{11} = \sqcup q_5 x 0 x 0 x$	$\delta(q_5, 0) = (q_5, 0, L)$
$C_{12} = q_5 \sqcup x 0 x 0 x$	$\delta(q_5, x) = (q_5, x, L)$
$C_{13} = \sqcup q_2 x 0 x 0 x$	$\delta(q_5, \sqcup) = (q_2, \sqcup, R)$
$C_{14} = \sqcup x q_2 0 x 0 x$	$\delta(q_2, x) = (q_2, x, R)$
$C_{15} = \sqcup x x q_3 x 0 x$	$\delta(q_2, 0) = (q_3, x, R)$
$C_{16} = \sqcup x x x q_3 0 x$	$\delta(q_3, x) = (q_3, x, R)$
$C_{17} = \sqcup x x x 0 q_4 x$	$\delta(q_3, 0) = (q_4, 0, R)$
$C_{18} = \sqcup x x x 0 x q_4 \sqcup$	$\delta(q_4, x) = (q_4, x, R)$
$C_{19} = \sqcup x x x 0 x \sqcup q_{\text{reject}} \sqcup$	$\delta(q_4, \sqcup) = (q_{\text{reject}}, \sqcup, R)$

Resultado: rechaza. Tras la primera pasada quedan tres ceros sin tachar; en la segunda pasada el patrón de paridad cambia y M_2 detecta la cantidad impar al llegar al blanco final estando en q_4 .

Verificación

Como $1 = 2^0$ y $2 = 2^1$ son potencias de 2 pero 3 y 6 no lo son,

$$0 \in A, \quad 00 \in A, \quad 000 \notin A, \quad 000000 \notin A,$$

en concordancia con los resultados obtenidos.

3. Ejercicio 3: deciders por marcado y emparejamiento

Enunciado

Dar definiciones de máquinas de Turing para decidir los siguientes lenguajes:

- (a) $L_1 = \{w \in \{0, 1\}^* \mid \#_0(w) = \#_1(w)\}$.
- (b) $L_2 = \{w \in \{0, 1\}^* \mid \#_0(w) = 2 \cdot \#_1(w)\}$.

Estrategia general

Usamos la técnica de *marcado y emparejamiento* ilustrada en los Ejemplos 3.9 y 3.11 de Sipser. El alfabeto de cinta es $\Gamma = \{0, 1, x, y, \sqcup\}$, donde x marca un 0 ya emparejado, y marca un 1 ya emparejado, y $\sqcup$ es el blanco. Cada iteración toma un símbolo no marcado y busca su contraparte (uno o dos 0s, según el caso).

(a) MT que decide L_1

Descripción a nivel implementación

M_{eq} = “Con entrada w :

1. Volver al inicio de la cinta.
2. Recorrer la cinta hacia la derecha hasta el primer símbolo no marcado (0 o 1).
3. Si no hay ninguno (sólo x , y y blancos), *aceptar*.
4. Si el símbolo es 0, marcarlo como x y buscar a su derecha el primer 1 no marcado. Si lo encuentra, marcarlo como y y volver a (2). Si llega a un blanco sin encontrarlo, *rechazar*.
5. Si el símbolo es 1, marcarlo como y y buscar a su derecha el primer 0 no marcado. Si lo encuentra, marcarlo como x y volver a (2). Si llega a un blanco sin encontrarlo, *rechazar*.”

Descripción formal

$$M_{eq} = (Q, \Sigma, \Gamma, \delta, q_{seek}, q_{accept}, q_{reject}),$$

con $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, x, y, \sqcup\}$ y $Q = \{q_{seek}, q_{find1}, q_{find0}, q_{back}, q_{accept}, q_{reject}\}$.

Función de transición.

$$\begin{aligned} \delta(q_{seek}, x) &= (q_{seek}, x, R), & \delta(q_{seek}, y) &= (q_{seek}, y, R), \\ \delta(q_{seek}, 0) &= (q_{find1}, x, R), & \delta(q_{seek}, 1) &= (q_{find0}, y, R), \\ \delta(q_{seek}, \sqcup) &= (q_{accept}, \sqcup, R). \end{aligned}$$

$$\begin{aligned} \delta(q_{find1}, a) &= (q_{find1}, a, R) \quad \text{para } a \in \{0, x, y\}, \\ \delta(q_{find1}, 1) &= (q_{back}, y, L), \quad \delta(q_{find1}, \sqcup) = (q_{reject}, \sqcup, R). \end{aligned}$$

$$\begin{aligned} \delta(q_{find0}, a) &= (q_{find0}, a, R) \quad \text{para } a \in \{1, x, y\}, \\ \delta(q_{find0}, 0) &= (q_{back}, x, L), \quad \delta(q_{find0}, \sqcup) = (q_{reject}, \sqcup, R). \end{aligned}$$

$$\begin{aligned} \delta(q_{back}, a) &= (q_{back}, a, L) \quad \text{para } a \in \{0, 1, x, y\}, \\ \delta(q_{back}, \sqcup) &= (q_{seek}, \sqcup, R). \end{aligned}$$

Correctitud y terminación

Si M_{eq} acepta: en cada iteración completa se marcaron en pareja un 0 y un 1. La aceptación ocurre cuando q_{seek} lee un blanco, indicando que ya no hay símbolos sin marcar. Como las marcas son en pareja, $\#_0(w) = \#_1(w)$.

Si $\#_0(w) = \#_1(w)$: en cada iteración, al elegir un símbolo no marcado, la igualdad de cantidades garantiza que existe la contraparte. Así M_{eq} no entra en q_{reject} y termina aceptando al consumir todos los símbolos.

Terminación: cada iteración completa marca al menos dos símbolos nuevos. Como $|w| < \infty$, tras a lo sumo $\lfloor |w|/2 \rfloor$ iteraciones la máquina se detiene.

(b) MT que decide L_2

Descripción a nivel implementación

$M_{2:1}$ = “Con entrada w :

1. Volver al inicio de la cinta.
2. Buscar el primer 1 no marcado.
3. Si no hay 1 sin marcar, pasar a la fase final:
 - recorrer la cinta y verificar que tampoco haya ningún 0 sin marcar;
 - si no queda ninguno, aceptar; en caso contrario, rechazar.
4. Si se encontró un 1, marcarlo como y .
5. Volver al inicio y buscar el primer 0 sin marcar; si no existe, rechazar; si existe, marcarlo como x .
6. Continuar a la derecha y buscar un segundo 0 sin marcar; si no existe, rechazar; si existe, marcarlo como x .
7. Volver al inicio y volver a (2).”

Descripción formal

$$M_{2:1} = (Q, \Sigma, \Gamma, \delta, q_{scan1}, q_{accept}, q_{reject}),$$

con $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, x, y, \sqcup\}$ y

$$Q = \{q_{scan1}, q_{back1}, q_{find0a}, q_{find0b}, q_{back}, q_{backf}, q_{check0}, q_{accept}, q_{reject}\}.$$

Función de transición.

$$\begin{aligned}\delta(q_{scan1}, a) &= (q_{scan1}, a, R) \quad \text{para } a \in \{0, x, y\}, \\ \delta(q_{scan1}, 1) &= (q_{back1}, y, L), \\ \delta(q_{scan1}, \sqcup) &= (q_{backf}, \sqcup, L).\end{aligned}$$

$$\begin{aligned}\delta(q_{back1}, a) &= (q_{back1}, a, L) \quad \text{para } a \in \{0, 1, x, y\}, \\ \delta(q_{back1}, \sqcup) &= (q_{find0a}, \sqcup, R).\end{aligned}$$

$$\begin{aligned}\delta(q_{\text{find0a}}, a) &= (q_{\text{find0a}}, a, R) \quad \text{para } a \in \{1, x, y\}, \\ \delta(q_{\text{find0a}}, 0) &= (q_{\text{find0b}}, x, R), \quad \delta(q_{\text{find0a}}, \sqcup) = (q_{\text{reject}}, \sqcup, R).\end{aligned}$$

$$\begin{aligned}\delta(q_{\text{find0b}}, a) &= (q_{\text{find0b}}, a, R) \quad \text{para } a \in \{1, x, y\}, \\ \delta(q_{\text{find0b}}, 0) &= (q_{\text{back}}, x, L), \quad \delta(q_{\text{find0b}}, \sqcup) = (q_{\text{reject}}, \sqcup, R).\end{aligned}$$

$$\begin{aligned}\delta(q_{\text{back}}, a) &= (q_{\text{back}}, a, L) \quad \text{para } a \in \{0, 1, x, y\}, \\ \delta(q_{\text{back}}, \sqcup) &= (q_{\text{scan1}}, \sqcup, R).\end{aligned}$$

$$\begin{aligned}\delta(q_{\text{backf}}, a) &= (q_{\text{backf}}, a, L) \quad \text{para } a \in \{0, 1, x, y\}, \\ \delta(q_{\text{backf}}, \sqcup) &= (q_{\text{check0}}, \sqcup, R).\end{aligned}$$

$$\begin{aligned}\delta(q_{\text{check0}}, x) &= (q_{\text{check0}}, x, R), \quad \delta(q_{\text{check0}}, y) = (q_{\text{check0}}, y, R), \\ \delta(q_{\text{check0}}, 0) &= (q_{\text{reject}}, 0, R), \quad \delta(q_{\text{check0}}, 1) = (q_{\text{reject}}, 1, R), \\ \delta(q_{\text{check0}}, \sqcup) &= (q_{\text{accept}}, \sqcup, R).\end{aligned}$$

Correctitud y terminación

Si $M_{2:1}$ acepta: en cada ciclo completado se marcó exactamente un 1 y dos 0s. La fase final verifica que no quedan ni 1s ni 0s sueltos. Llamando k al número de ciclos completados, $\#_1(w) = k$ y $\#_0(w) = 2k$, por lo cual $\#_0(w) = 2 \#_1(w)$.

Si $\#_0(w) = 2 \#_1(w)$: al iniciar el ciclo $i + 1$ restan $\#_1(w) - i$ unos y $2(\#_1(w) - i)$ ceros sin marcar. Mientras $\#_1(w) - i \geq 1$, hay al menos dos 0s para emparejar al 1 recién marcado, así que el ciclo completa sin rechazar. Tras $\#_1(w)$ ciclos no quedan símbolos sin marcar y la fase final acepta.

Terminación: cada ciclo marca un 1 y dos 0s, o rechaza. Como $|w| < \infty$, tras a lo sumo $\#_1(w)$ ciclos la máquina llega a la fase final.

Conclusión

L_1 y L_2 son lenguajes decidibles.

Las MT M_{eq} y $M_{2:1}$ son deciders explícitos. Sus implementaciones en Python (archivo `ejercicio4_mt_teorico.py`) verifican automáticamente la equivalencia entre la descripción formal y el comportamiento ejecutado.

4. Ejercicio 4: cinta doblemente infinita

Enunciado

Una máquina de Turing con *cinta doblemente infinita* es una MT cuya cinta es infinita tanto a la izquierda como a la derecha. Mostrar que estas máquinas reconocen los mismos lenguajes que las MT estándar (Problema 3.11 de Sipser).

Modelos

- **MT estándar** (TM_{std}): una sola cinta infinita hacia la derecha, con extremo izquierdo. Si el cabezal intenta cruzar el extremo izquierdo, se queda en su lugar.
- **MT doblemente infinita** (TM_2): una sola cinta indexada por $\mathbb{Z}$. La entrada $w = w_0 \cdots w_{n-1}$ ocupa las celdas $0, 1, \dots, n-1$, el resto contiene blancos, y el cabezal arranca en la celda 0.

Teorema 4.1. $\mathcal{L}(\text{TM}_{\text{std}}) = \mathcal{L}(\text{TM}_2)$.

Demostración. Probamos las dos inclusiones por separado.

($\subseteq$) Sea $M \in \text{TM}_{\text{std}}$. Construimos $N \in \text{TM}_2$ que simula a M reservando un marcador $\vdash \in \Gamma_N \setminus \Gamma_M$ en la celda -1 . La descripción:

$N =$ “Con entrada w :

1. Escribir $\vdash$ en la celda -1 .
2. Simular paso a paso a M sobre las celdas $0, 1, \dots$
3. Si la transición indica mover a la izquierda y la celda actual es $\vdash$, N se queda en su lugar (replicando el comportamiento de borde izquierdo de M).
4. Cuando M acepta o rechaza, N acepta o rechaza.”

Por simulación paso a paso, $L(N) = L(M)$.

($\supseteq$) Sea $B \in \text{TM}_2$. Construimos $S \in \text{TM}_{\text{std}}$ que “dobla la cinta sobre sí misma”.

Codificación. Definimos la biyección $e : \mathbb{Z} \rightarrow \mathbb{N}$:

$$e(i) = \begin{cases} 2i & i \geq 0, \\ -2i - 1 & i < 0. \end{cases}$$

Esto manda $0 \mapsto 0, 1 \mapsto 2, 2 \mapsto 4, \dots, -1 \mapsto 1, -2 \mapsto 3, \dots$. La celda $e(i)$ de S guarda el símbolo que B tiene en la celda i . Para indicar la posición del cabezal de B , usamos un alfabeto extendido $\Gamma_S = \Gamma \cup \dot{\Gamma}$ con $\dot{\Gamma} = \{\dot{a} : a \in \Gamma\}$. Mantenemos la invariante: en todo momento exactamente una celda de S tiene un símbolo punteado, y corresponde a la posición del cabezal de B .

Inicialización. S reorganiza $w = w_0 \cdots w_{n-1}$ en las celdas $0, 2, \dots, 2(n-1)$, intercala blancos en las celdas $1, 3, \dots$, y marca la celda 0 como $\dot{w}_0$.

Simulación de un paso. Si S lee $\dot{a}$ en la celda $k = e(h)$, B está en estado q con $\delta(q, a) = (p, b, D)$, entonces S :

1. escribe b (sin marca) en la celda k ;
2. calcula $k' = e(h')$ con $h' = h \pm 1$ según D ;

3. marca la celda k' .

Las fórmulas explícitas para k' se obtienen examinando paridades:

$$k'|_{D=R} = \begin{cases} k+2 & k \text{ par}, \\ 0 & k = 1, \\ k-2 & k > 1 \text{ impar}, \end{cases} \quad k'|_{D=L} = \begin{cases} 1 & k = 0, \\ k-2 & k > 0 \text{ par}, \\ k+2 & k \text{ impar}. \end{cases}$$

S sólo se mueve a lo sumo dos celdas para reubicar la marca.

Correctitud. Por inducción en el número de pasos, después de simular el paso t -ésimo la cinta de S codifica fielmente la de B y la única celda marcada coincide con el cabezal de B . Por lo tanto, S acepta sii B acepta y $L(S) = L(B)$. $\square$

Observacion. Este resultado es un caso más de la *robustez* del modelo de MT (Sipser, pág. 176): variantes “naturales” del modelo no cambian la clase de lenguajes reconocidos. La codificación e es la misma idea que la biyección $\mathbb{N} \cong \mathbb{Z}$ usada en la teoría de la cátedra (Diag.pdf); aquí se aplica como herramienta de simulación.

5. Ejercicio 5: subconjunto infinito decidable

Enunciado

Demostrar que cualquier lenguaje infinito que es reconocible por una máquina de Turing tiene un subconjunto infinito que es decidable (Problema 3.19 de Sipser).

Proposición 5.1. Sea $A \subseteq \Sigma^*$ infinito y Turing-reconocible. Entonces existe $B \subseteq A$ tal que B es infinito y decidable.

Demostración. Como A es Turing-reconocible, por el Teorema 3.21 de Sipser existe un enumerador E que enumera A :

$$A = \{s : E \text{ imprime } s \text{ en algún momento}\}.$$

Fijamos sobre Σ^* el orden *shortlex* (primero por longitud, luego lexicográfico): $u < v$ si $|u| < |v|$, o si $|u| = |v|$ y $u <_{\text{lex}} v$. Es un orden total efectivo y bien fundado.

Sucesión creciente en A . Definimos $(b_i)_{i \geq 1}$ inductivamente:

- b_1 = primera palabra que imprime E .
- Dado b_i , sea b_{i+1} la primera palabra que imprime E y cumple $b_{i+1} > b_i$ en shortlex.

La definición es efectiva porque, para cada i :

- podemos simular E paso a paso;
- como A es infinito y $b_i \in A$, hay infinitas palabras de A mayores que b_i en shortlex, y E imprime cada una en tiempo finito;
- por lo tanto, en algún paso E imprime una mayor que b_i , que tomamos como b_{i+1} .

Definimos $B = \{b_1, b_2, b_3, \dots\}$. Por construcción, cada $b_i \in A$, $B \subseteq A$, y la sucesión $b_1 < b_2 < \dots$ es estrictamente creciente, de modo que B es infinito.

Decisión de B . Construimos un decidor D_B :

$D_B =$ “Con entrada x :

1. Generar $b_1, b_2, \dots$ de a uno usando el procedimiento anterior.
2. Detenerse en el primer b_k con $b_k \geq x$ en shortlex.
3. Si $b_k = x$, aceptar; si $b_k > x$, rechazar.”

Correctitud. Si $x \in B$, existe k con $b_k = x$; al alcanzar ese k el algoritmo acepta. Si $x \notin B$, sea k el primer $b_k \geq x$; necesariamente $b_k > x$, y el algoritmo rechaza.

Terminación. Bajo shortlex, $\{y \in \Sigma^* : y \leq x\}$ es finito. Como (b_i) es estrictamente creciente, en finitos pasos se alcanza $b_k \geq x$. Cada b_i se calcula en tiempo finito. Luego D_B siempre se detiene.

Por lo tanto B es infinito y decidable, y $B \subseteq A$. □

Observación. Es interesante notar que un lenguaje reconocible no es en general decidable (como muestra A_{TM} , Capítulo 4 y [Diag.pdf](#)), pero *siempre* se puede aislar dentro de él una porción decidable que conserve la cardinalidad. En particular, un lenguaje admite enumeración creciente efectiva si y sólo si es decidable (Problema 3.18 de Sipser).

6. Ejercicio 6: MT con input read-only

Enunciado

Mostrar que una MT con una sola cinta que no puede escribir sobre la parte de la cinta que contiene el input sólo puede reconocer lenguajes regulares (Problema 3.20 de Sipser).

Modelo y proposición

Adoptamos la interpretación estándar: la MT M tiene una sola cinta y, sobre las celdas que contienen el input, las transiciones $\delta(q, a)$ *deben* escribir el mismo símbolo a . Asumimos por comodidad que la zona del input está delimitada por marcadores $\vdash$ (extremo izquierdo) y $\dashv$ (extremo derecho), y que M nunca cruza esos límites.

Bajo esta hipótesis, la cinta es de *sólo lectura* sobre el input, las transiciones se reducen a $\delta(q, a) = (p, a, D)$ con $D \in \{L, R\}$, y el modelo coincide con un **2DFA** (autómata finito determinista de dos vías).

Proposición 6.1. *Todo lenguaje reconocido por una MT M que no escribe sobre el input es regular.*

Demostración. Construimos un 2DFA equivalente. Sean

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

y $A = (Q, \Sigma, \delta_A, q_0, \{q_{\text{accept}}\})$ con $\delta_A : Q \times (\Sigma \cup \{\vdash, \dashv\}) \rightarrow Q \times \{L, R\}$ definida por:

- Si $\delta(q, a) = (p, a, D)$ con $a \in \Sigma$, entonces $\delta_A(q, a) = (p, D)$.
- En $\vdash$ y $\dashv$, δ_A reproduce el comportamiento de borde de M (en particular, A no cruza los marcadores).

- Cuando M entra en q_{accept} o q_{reject} , A entra en esos estados.

Como M no altera la cinta de entrada, cada paso de M se reproduce uno a uno como un paso de A . Luego $L(M) = L(A)$.

Por el teorema clásico de **Rabin–Scott / Shepherdson** (1959), los lenguajes reconocidos por 2DFA son exactamente los regulares. Por lo tanto $L(A)$ es regular y, por la igualdad, $L(M)$ también lo es. $\square$

Observacion. Este resultado ilumina por qué la *escritura sobre la cinta* es esencial para la potencia de las MT. Sin esa capacidad, las MT colapsan a la clase regular y pierden la posibilidad de reconocer lenguajes tan básicos como $\{0^n 1^n : n \geq 0\}$, que requieren memoria proporcional a la entrada.

7. Parte de programación

La parte de programación pide implementar las tres versiones de MT (cinta única e infinita a derecha, doblemente infinita, y multicinta), las MT del Ejercicio 3 teórico, y los algoritmos de traducción entre los modelos. Las implementaciones están disponibles en el repositorio (`maquina_turing.py`, `maquina_turing_doble_infinita.py`, `maquina_turing_multicinta.py`, `ejercicio{1,2,3,4}*.py`) y han sido testeadas contra los predicados matemáticos correspondientes.

A continuación documentamos los algoritmos de traducción, que constituyen la parte conceptualmente más relevante.

Notación

- TM_1 : MT con una cinta infinita hacia la derecha.
- TM_2 : MT con una cinta doblemente infinita.
- TM_k : MT con $k \geq 1$ cintas, cada una infinita hacia la derecha.

A) $TM_1 \rightarrow TM_2$

Dada $M \in TM_1$, construir $N \in TM_2$ con $L(N) = L(M)$.

1. Reservar $\vdash \in \Gamma_N \setminus \Gamma_M$ como marcador de borde izquierdo.
2. En la inicialización, N escribe $\vdash$ en la celda -1 .
3. Cada transición de M se traslada literalmente a N . Las transiciones de movimiento a la izquierda sobre la celda con $\vdash$ dejan el cabezal en su lugar, replicando el comportamiento de borde izquierdo de TM_1 .
4. Estados de aceptación y rechazo se preservan.

(Construcción detallada en la primera parte del Ejercicio 4.)

B) $TM_2 \rightarrow TM_1$

Dada $B \in TM_2$, construir $S \in TM_1$ con $L(S) = L(B)$.

1. Codificar $i \in \mathbb{Z}$ con $e(i) = 2i$ si $i \geq 0$, $e(i) = -2i - 1$ si $i < 0$.
2. En S , la celda $e(i)$ guarda el símbolo de B en la celda i .

3. Marcar exactamente una celda con un símbolo punteado para indicar la posición del cabezal de B .
4. Inicialización: reorganizar la entrada en las celdas $0, 2, \dots, 2(n-1)$, intercalar blancos en las impares, marcar la celda 0.
5. Cada paso de B se simula con escritura local, cálculo de la nueva posición $e(i \pm 1)$ y reubicación de la marca.

(Construcción detallada en la segunda parte del Ejercicio 4.)

C) $\text{TM}_1 \rightarrow \text{TM}_k$

Dada $M \in \text{TM}_1$ y $k \geq 1$, construir $N \in \text{TM}_k$ con $L(N) = L(M)$:

1. Usar la cinta 1 de N para simular la cinta de M .
2. Mantener las cintas $2, 3, \dots, k$ en blanco y sin movimiento.
3. Cada $\delta_M(q, a) = (p, b, D)$ se traduce a una transición de TM_k que solo afecta la cinta 1.

D) $\text{TM}_k \rightarrow \text{TM}_1$

Es la construcción del Teorema 3.13 de Sipser. Dada $K \in \text{TM}_k$, construir $U \in \text{TM}_1$:

1. Codificar las k cintas de K en una sola cinta de U con delimitadores: $\# u_1 \# u_2 \# \dots \# u_k \#$.
2. En cada bloque, marcar con un símbolo punteado el símbolo bajo el cabezal de la cinta correspondiente.
3. Simular un paso de K en tres fases:
 - *Lectura*: U recorre la cinta y memoriza en su control finito los k símbolos marcados.
 - *Cálculo*: consulta δ_K con esos símbolos y el estado actual.
 - *Escritura/movimiento*: segunda pasada para actualizar los símbolos marcados y mover las marcas. Si una marca debe avanzar más allá del fin de su bloque, U expande el bloque desplazando todo a la derecha.

Por simulación paso a paso, $L(U) = L(K)$.

E) Traducciones restantes

Por composición:

- $\text{TM}_2 \rightarrow \text{TM}_k$: aplicar B y luego C.
- $\text{TM}_k \rightarrow \text{TM}_2$: aplicar D y luego A.

Conclusión

$$\mathcal{L}(\text{TM}_1) = \mathcal{L}(\text{TM}_2) = \mathcal{L}(\text{TM}_k) \quad \text{para todo } k \geq 1.$$

Esta equivalencia es la *robustez* del modelo de MT y unifica las tres versiones implementadas en código.

Conclusiones

A lo largo del práctico hemos consolidado los conceptos centrales del Capítulo 3 de Sipser:

- El Ejercicio 2 muestra cómo opera una MT concreta paso a paso.
- El Ejercicio 3 ilustra la técnica de *marcado y emparejamiento* para construir deciders de lenguajes contextuales.
- Los Ejercicios 4 y 6 estudian variantes del modelo: la cinta doblemente infinita preserva la potencia, mientras que la restricción de no escribir sobre el input la reduce a la clase regular.
- El Ejercicio 5 establece un puente entre reconocibilidad y decidibilidad: dentro de cualquier lenguaje reconocible infinito hay un fragmento decidable infinito.
- La parte de programación cierra el círculo conectando la formalización de los modelos con su simulación en código y los algoritmos de traducción.

En conjunto, los resultados refuerzan la *tesis de Church–Turing*: el modelo de MT (en cualquiera de sus variantes razonables) captura efectivamente la noción intuitiva de algoritmo, y proveen los cimientos sobre los que se construyen los conceptos de *indecidibilidad* y *reducción* desarrollados en el Práctico 02.